OpKAN: A Physics-Informed Kolmogorov-Arnold Network with Dual-Process LLM-Driven Topology for Regime-Adaptive Options Pricing

LiuClaw Agent & Engineering Team
`github.com/RicardoLaMo/OpKAN`

March 2026

Abstract

We present **OpKAN**, an architecture that joins physics-informed deep learning with real-time large language model (LLM) guidance to solve the Heston stochastic volatility partial differential equation (PDE) for options pricing. The system is built on three interlocking layers. First, a Physics-Informed Kolmogorov-Arnold Network (PI-KAN) replaces the standard multi-layer perceptron (MLP) used in conventional PINNs with a grid of learnable B-spline edge activations, guaranteeing the C^2 continuity required by the Heston PDE's second-order cross-derivatives. Second, a Gaussian Hidden Markov Model (HMM) with walk-forward, realized-volatility–sorted regime labeling provides real-time market state signals without look-ahead bias. Third, **LiuClaw**, a dual-process reasoning agent inspired by Kahneman's System 1/System 2 cognitive architecture [Kahneman, 2011], bridges qualitative market narrative to quantitative topology: a fast reflexive branch (Qwen 2.5-7B at < 50 ms) handles high-frequency pruning and learning-rate adjustments, while a slow strategic branch (Qwen 2.5-32B at 200–500 ms) issues C2-symbolic `REPLACE` mutations grounded in PDE geometry. All LLM-generated symbolic expressions pass an AST allowlist validator before evaluation, preventing code-injection attacks that we demonstrate are otherwise trivially exploitable. Mutations are executed atomically with layer-snapshot rollback, and all decisions are serialized through a Pydantic-enforced domain-specific language (DSL) that makes every agent action auditable and replayable.

On a curated 15-case protocol benchmark, the LiuClaw agent achieves 1.00 execution, translation, and intent-match rates, 0.92 semantic-signature match, and 0.75 group-level paraphrase invariance. On a harder document-to-gold benchmark against human-authored DSL, semantic agreement reaches 0.75 while exact string agreement is 0.375, exposing the normalization frontier that separates protocol viability from full analyst automation. Quantitatively, the SPIKAN plain-anchor IV-residual hybrid achieves the lowest replay price MAE in transition (1.46) and stress (0.42) regimes, while raw KAN/SPIKAN fails all five financial monotonicity checks without an explicit semantic controller. Engineering throughput on NVIDIA H200 hardware reaches **26,300** samples/sec under a 4-second asynchronous LLM decision cycle.

The central claim is protocol-first: language acts as a constrained, auditable interoperability layer between unstructured market narrative and specialist quantitative execution. OpKAN is not a replacement for pricing formulas; it is a regime-aware reasoning envelope that makes topology mutations mathematically grounded, semantically interpretable, and operationally safe.

Contents

1 Introduction **4**

2 Related Work **5**
2.1 Classical Option Pricing and Stochastic Volatility . . . 5
2.2 Physics-Informed Neural Networks . . . 5
2.3 Kolmogorov-Arnold Networks . . . 5
2.4 Machine Learning for Options and Implied Volatility . . . 6
2.5 LLM Agents for Quantitative Finance . . . 6
2.6 Monotone and Interpretable Learned Models . . . 6
2.7 Dual-Process Cognition and AI . . . 6
2.8 Hidden Markov Models for Market Regimes . . . 6

3 Mathematical Foundations **7**
3.1 Kolmogorov-Arnold Representation and B-Splines . . . 7
3.2 The Heston Stochastic Volatility Model . . . 7
3.3 Boundary Conditions . . . 7
3.4 PI-KAN Loss Formulation . . . 8
3.5 HMM Regime Detection . . . 8

4 System Architecture **8**
4.1 Overview . . . 8
4.2 Data Pipeline and Feature Extraction . . . 8
4.3 PI-KAN Model . . . 9
4.4 LiuClaw: A Dual-Process Agent . . . 9
4.4.1 System 1 – Fast Thinking . . . 9
4.4.2 System 2 – Slow Thinking . . . 9
4.4.3 Domain-Specific Language . . . 10
4.5 Asynchronous Coordination . . . 10

5 Security and Correctness **11**
5.1 The eval() Injection Vulnerability . . . 11
5.2 AST Allowlist Validator . . . 11
5.3 ZeroEdge: Correct PRUNE Semantics . . . 12
5.4 Atomic Mutation Rollback . . . 12
5.5 HMM Label Stability . . . 12

6 Experimental Design **12**
6.1 Research Questions . . . 12
6.2 Protocol Benchmark Tiers . . . 13
6.3 Pricing Benchmarks . . . 13
6.4 Model Families Compared . . . 13
6.5 Engineering Stress Test . . . 14

7 Results **14**
7.1 LLM→DSL Protocol Results . . . 14
7.2 Semantic Protocol Stability . . . 14
7.3 Document-to-Gold Agreement . . . 15
7.4 Regime-Action Coherence . . . 15
7.5 Quantitative Pricing Results . . . 16
7.6 Cross-Asset Transfer . . . 16
7.7 Engineering Performance . . . 17

8 Discussion **17**
8.1 What the Results Support 17
8.2 What the Results Do Not Support 17
8.3 The Right Comparison Class 18

9 Limitations **18**

10 Future Work **19**

11 Conclusion **19**

1 Introduction

Modern options desks face two structurally different problems that are almost always addressed by separate tools. The first is *quantitative*: how should a model produce prices, implied volatilities (IVs), and Greeks that remain consistent with no-arbitrage conditions and financial boundary structure? The second is *informational*: how should qualitative, unstructured inputs — macro news, research reports, FOMC commentary, supply-shock narratives — influence that pricing model in a controlled, auditable way?

Classical approaches answer the first problem well. Black–Scholes [Black and Scholes, 1973] and Merton [Merton, 1973] provide closed-form anchors with consistent Greeks and well-defined PDE interpretations. The Heston stochastic-volatility model [Heston, 1993] extends this to capture the leverage effect and volatility clustering observed empirically. But closed-form models are structurally rigid; they adapt to regime change only through manual parameter recalibration, which is expensive and subject to the analyst's availability and judgment.

Machine learning partially bridges the gap. Neural networks can fit option surfaces non-parametrically [Hutchinson et al., 1994], and physics-informed neural networks (PINNs) [Raissi et al., 2019] embed PDE residuals directly into the loss function so that the learned function is constrained to satisfy the governing equation. Yet these systems share a critical weakness: they assume that the relevant information has already been pre-processed into structured numerical features. In practice, the most market-moving signals — a geopolitical announcement, a central bank pivot, an inventory squeeze report — arrive first as natural language, and the latency of converting them into structured overrides is precisely where systematic edge is lost.

OpKAN addresses both problems through a three-tier hierarchy, illustrated in Figure 1:

1. **Quantitative core.** A PI-KAN solves the Heston PDE over randomly sampled interior collocation points, with three sets of boundary collocation points enforcing the terminal payoff condition and the asymptotic option price limits.

2. **Regime detector.** A Gaussian HMM runs walk-forward on five market microstructure features (log returns, realized volatility, implied volatility, IV–RV spread, IV momentum), producing a stable, relabeled regime signal.

3. **LiuClaw agent.** A dual-process LLM agent reads HMM output and KAN edge statistics, then issues either fast reflexive maintenance decisions (pruning near-zero edges, adjusting learning rate) or slow strategic decisions (replacing B-spline edges with interpretable symbolic functions, diagnosing regime shifts).

The central architectural insight is that *the LLM does not price options.* It functions as a constrained semantic compiler: it ingests unstructured text and converts it into a finite, typed, Pydantic-validated DSL that drives the quantitative execution layer. This design separates concerns cleanly. Language understanding lives in the LLM; mathematical correctness lives in the PINN; auditability lives in the DSL. No component is asked to do more than it is good at.

This paper makes the following contributions:

1. A full PI-KAN implementation using genuine Cox–de Boor B-spline basis functions in place of prior placeholder activations, with C^2 continuity verified by second-order autograd.

2. A dual-process LiuClaw agent architecture that separates high-frequency reflexive maintenance from low-frequency strategic topology surgery.

3. A formally specified, Pydantic-enforced DSL with an AST allowlist security layer that provably blocks code-injection attacks while permitting a practical set of smooth, differentiable symbolic functions.

4. An atomic mutation rollback mechanism that restores all affected layer modules on any failure, preventing partially-mutated model state.

5. A comprehensive evaluation spanning five protocol benchmark tiers, regime coherence tests, live and replay pricing benchmarks, and cross-asset transfer experiments.

6. An honest enumeration of what the current evidence supports and what it does not.

The paper is organized as follows. Section 2 reviews the relevant literature. Section 3 develops the mathematical foundations. Section 4 describes the full system architecture. Section 5 discusses the security and correctness properties. Section 6 presents the experimental design. Section 7 reports results. Section 8 interprets them. Section 9 states limitations. Section 10 outlines the next architecture steps. Section 11 concludes.

2 Related Work

2.1 Classical Option Pricing and Stochastic Volatility

The Black–Scholes–Merton framework [Black and Scholes, 1973, Merton, 1973] remains the canonical structural anchor for options pricing. It is structurally disciplined, produces consistent Greeks, and provides a well-defined PDE whose boundary conditions and terminal payoff structure serve as hard constraints. The Heston model [Heston, 1993] introduces a mean-reverting stochastic variance process that captures the volatility smile and the leverage effect observed empirically across equity and commodity derivatives. For OpKAN, the Heston PDE plays the role of a physics prior that regularizes the learned KAN surface; we do not discard the formula layer but embed it as a constraint.

Finite difference methods (FDM) [Wilmott et al., 1993] and Monte Carlo simulation solve the Heston PDE numerically and serve as evaluation baselines. Their limitation for our purposes is computational inflexibility: re-solving under regime-changed parameters is expensive, and neither approach naturally incorporates LLM-generated scenario overrides.

2.2 Physics-Informed Neural Networks

PINNs [Raissi et al., 2019] embed PDE residuals directly into the training loss, using automatic differentiation to evaluate the differential operator at randomly sampled interior collocation points. This approach converts the PDE boundary value problem into an unconstrained optimization, making it compatible with any differentiable architecture. For financial PDEs, PINNs have been applied to Black–Scholes [Horvath et al., 2021] and Heston dynamics, though prior work often uses standard MLPs whose ReLU activations have zero second derivatives, creating instability in the cross-derivative terms $\frac{\partial^2 V}{\partial S \partial v}$. OpKAN replaces the MLP with a KAN whose B-spline activations are genuinely C^2 continuous.

2.3 Kolmogorov-Arnold Networks

Kolmogorov's superposition theorem [Kolmogorov, 1957] proves that any multivariate continuous function can be represented as a composition of univariate continuous functions. Liu et al. [2024] operationalize this by placing learnable activation functions on *edges* rather than nodes, parameterized as B-spline expansions. Compared to MLPs, KANs offer more interpretable function representations, since each edge activation can be visualized independently, and they admit surgical replacement of individual edges with closed-form symbolic expressions discovered by symbolic regression or LLM reasoning. MonoKAN [Polo-Molina et al., 2024] extends KANs with certified monotonicity constraints, which is the direction OpKAN's future controller evolution points toward.

2.4 Machine Learning for Options and Implied Volatility

Hutchinson et al. [1994] pioneered nonparametric options pricing using neural networks. More recent approaches include neural SDE models, Gaussian process regression on IV surfaces, and deep calibration of stochastic volatility parameters [Hernandez, 2016]. A recurrent theme is that pure machine learning models fit structured data well but do not naturally ingest unstructured information. The random-forest baseline in our experiments is the canonical example: strong IV reconstruction on structured inputs, weak adaptation when qualitative regime signals must be incorporated without manual feature engineering.

2.5 LLM Agents for Quantitative Finance

Toolformer [Schick et al., 2023] and ReAct [Yao et al., 2023] enable LLMs to invoke external tools and APIs. For quantitative finance, several works have explored LLM-augmented trading systems, but most treat the LLM as a direct decision-maker or a generic tool-caller. OpKAN takes a different position: the LLM is a *semantic compiler*. Its output is not a price but a typed, validated DSL command that drives a specialist quantitative backend. This distinction matters operationally. A generic tool-call can fail silently or produce financially inconsistent outputs; a DSL command is validated by a Pydantic schema, restricted to a finite action vocabulary, and immediately executable against the current pricing state.

2.6 Monotone and Interpretable Learned Models

Financial monotonicity constraints — call prices increasing with volatility, decreasing with strike — are traditionally difficult to enforce in neural networks. Deep Lattice Networks [You et al., 2017] and MonoNet [Nguyen et al., 2023] provide partial monotonicity via lattice interpolation and projection heads, respectively. OpKAN's current architecture achieves regime-ordered behavior through an explicit semantic controller layered over the raw KAN/SPIKAN outputs, rather than through native architectural monotonicity. This is an honest current state: the controlled execution path is coherent; the raw learned function approximator is not.

2.7 Dual-Process Cognition and AI

The dual-process framework in cognitive science [Kahneman, 2011] describes two modes of human thought: System 1 (fast, automatic, low-effort) and System 2 (slow, deliberate, high-effort). Applied to AI agent design, this framework motivates architectures that separate latency-critical reactive loops from expensive deliberative planning. LiuClaw's fast/slow bifurcation is a direct implementation of this principle: System 1 handles sub-50ms reflexive maintenance (edge pruning, LR scheduling) that must not block the training loop; System 2 handles 200–500ms strategic topology reviews (symbolic replacement, regime diagnosis) that run asynchronously in a background thread.

2.8 Hidden Markov Models for Market Regimes

Gaussian HMMs are a standard tool for unsupervised market regime detection [Hamilton, 1989]. A persistent problem is state label instability: because HMM states are invariant to permutation, "state 0" in window t need not correspond to "state 0" in window $t+1$. We address this with a realized-volatility sorting pass after each window fit, ensuring that regime label 0 always maps to the lowest-volatility state across all windows, an approach grounded in the empirical regularity that realized volatility is the most discriminative regime feature in our feature set.

3 Mathematical Foundations

3.1 Kolmogorov-Arnold Representation and B-Splines

Definition 1 (Kolmogorov-Arnold Representation). *For any continuous function $f : [0,1]^n \to \mathbb{R}$, there exist continuous functions $\Phi_q : \mathbb{R} \to \mathbb{R}$ and $\phi_{q,p} : [0,1] \to \mathbb{R}$ such that:*

$$f(x_1, \ldots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right) \tag{1}$$

Liu et al. [2024] parameterize the edge functions $\phi_{q,p}$ as B-spline expansions. The k-th order B-spline basis functions $\{B_{i,k}\}$ over a knot sequence $\mathbf{t} = \{t_0, \ldots, t_m\}$ satisfy the Cox–de Boor recursion:

$$B_{i,0}(x) = \mathbf{1}[t_i \le x < t_{i+1}] \tag{2}$$

$$B_{i,k}(x) = \frac{x - t_i}{t_{i+k} - t_i} B_{i,k-1}(x) + \frac{t_{i+k+1} - x}{t_{i+k+1} - t_{i+1}} B_{i+1,k-1}(x) \tag{3}$$

For cubic splines ($k = 3$), this guarantees C^2 continuity across knot boundaries, which is the minimum regularity required for stable evaluation of second-order PDE operators. OpKAN uses $G = 5$ interior intervals with $k = 3$, yielding $G + k = 8$ basis functions per edge and $G + 2k + 1 = 12$ total knot points (including k padding knots on each side).

Each `BSplineEdge` in OpKAN implements the KAN-style activation:

$$\phi(x) = w_{\text{base}} \cdot \text{SiLU}(x) + w_{\text{spline}} \cdot \sum_{i=1}^{G+k} c_i B_{i,k}(x) \tag{4}$$

where $w_{\text{base}}, w_{\text{spline}} \in \mathbb{R}$ and $\{c_i\}$ are trainable parameters. The SiLU base ensures gradient flow at initialization; the spline term learns the domain-specific deviation.

3.2 The Heston Stochastic Volatility Model

The Heston model [Heston, 1993] specifies risk-neutral asset dynamics as:

$$dS_t = rS_t\, dt + \sqrt{v_t} S_t\, dW_t^S \tag{5}$$

$$dv_t = \kappa(\theta - v_t)\, dt + \sigma\sqrt{v_t}\, dW_t^v \tag{6}$$

where S_t is the asset price, v_t is the instantaneous variance, r is the risk-free rate, κ is the mean-reversion speed, θ is the long-run variance, σ is the vol-of-vol, and $\text{Corr}(dW^S, dW^v) = \rho\, dt$. The fair price $V(S, v, t)$ of a European call satisfies the Heston PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2} v S^2 \frac{\partial^2 V}{\partial S^2} + \rho\sigma v S \frac{\partial^2 V}{\partial S \partial v} + \frac{1}{2}\sigma^2 v \frac{\partial^2 V}{\partial v^2} + rS\frac{\partial V}{\partial S} + \kappa(\theta - v)\frac{\partial V}{\partial v} - rV = 0 \tag{7}$$

We use the default parameters: $r = 0.05$, $\kappa = 2.0$, $\theta = 0.04$, $\sigma = 0.3$, $\rho = -0.7$, $K = 100$, $T = 1$.

3.3 Boundary Conditions

The Heston PDE (7) is supplemented by three sets of boundary conditions that uniquely determine the European call price:

$$V(S, v, T) = \max(S - K, 0) \qquad \text{(terminal payoff)} \tag{8}$$

$$V(0, v, t) = 0 \qquad \text{(spot goes to zero)} \tag{9}$$

$$V(S, v, t) \approx S - Ke^{-r(T-t)} \qquad \text{as } S \to \infty \tag{10}$$

3.4 PI-KAN Loss Formulation

The Physics-Informed KAN minimizes a composite loss over two sets of collocation points: Ω (interior) and $\partial\Omega$ (boundary):

$$\mathcal{L} = \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\Omega}\left[|\mathcal{N}[V](\boldsymbol{x})|^2\right]}_{\mathcal{L}_{\text{PDE}}} + \lambda_{\text{BC}} \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\partial\Omega}\left[|V(\boldsymbol{x}) - V_{\text{target}}(\boldsymbol{x})|^2\right]}_{\mathcal{L}_{\text{BC}}} \tag{11}$$

where $\mathcal{N}[V]$ denotes the Heston differential operator evaluated term-by-term via `torch.autograd` with `create_graph=True` to enable second-order gradient computation. A critical implementation detail: the interior collocation points $\boldsymbol{x} = (S, v, t) \in [0, 200] \times [0, 0.5] \times [0, 1]$ are sampled *uniformly at random* each batch — they are not market data observations. Using market data features as collocation points would introduce non-uniform domain coverage that biases the PDE residual toward observed market states, constituting a subtle form of look-ahead bias that prior implementations of this system contained.

3.5 HMM Regime Detection

The regime detector fits a Gaussian HMM with diagonal covariance to a 5-dimensional feature vector $\boldsymbol{f}_t = (\ell_t, \hat{\sigma}_t, \sigma_t^{\text{IV}}, \Delta_t, \mu_t^{\text{IV}})$ comprising log-returns, realized volatility, implied volatility, IV–RV spread, and IV momentum. To maintain stable labels under walk-forward inference, we apply a post-fit state permutation:

$$\pi = \text{argsort}\left(\boldsymbol{\mu}_{:,1}\right), \quad \text{remap}[s] = \pi^{-1}[s] \tag{12}$$

where $\boldsymbol{\mu}_{:,1}$ is the vector of per-state mean realized volatilities. This ensures that label 0 always corresponds to the low-volatility regime and label 1 to the high-volatility regime across all rolling windows. Without this relabeling, a standard rolling HMM produces regime signals that are statistically meaningless due to random state permutation at each refit.

4 System Architecture

4.1 Overview

Figure 1 shows the four-tier information flow. Raw OPRA-format market data enters the data pipeline; a feature extractor produces the HMM input; the HMM outputs a regime label; the PI-KAN and the LiuClaw agent operate asynchronously on separate threads, with the agent reading KAN state and writing mutation decisions through a bounded thread-safe queue.

4.2 Data Pipeline and Feature Extraction

The data pipeline ingests OPRA-format CSV files via `src/data/parser.py`, which performs cleaning, implied volatility back-calculation via Newton–Raphson bisection, and augmentation. The feature extractor (`src/features/extractor.py`) derives five time-series features from the spot price and IV columns:

- $\ell_t = \log(S_t/S_{t-1})$ (log-return)
- $\hat{\sigma}_t = \text{std}(\ell_{t-W:t}) \cdot \sqrt{252 \times 6.5 \times 60}$ (annualized realized vol, $W = 20$)
- σ_t^{IV} (implied volatility from the options chain)
- $\Delta_t = \sigma_t^{\text{IV}} - \hat{\sigma}_t$ (IV–RV spread)
- $\mu_t^{\text{IV}} = \sigma_t^{\text{IV}} - \sigma_{t-1}^{\text{IV}}$ (IV momentum)

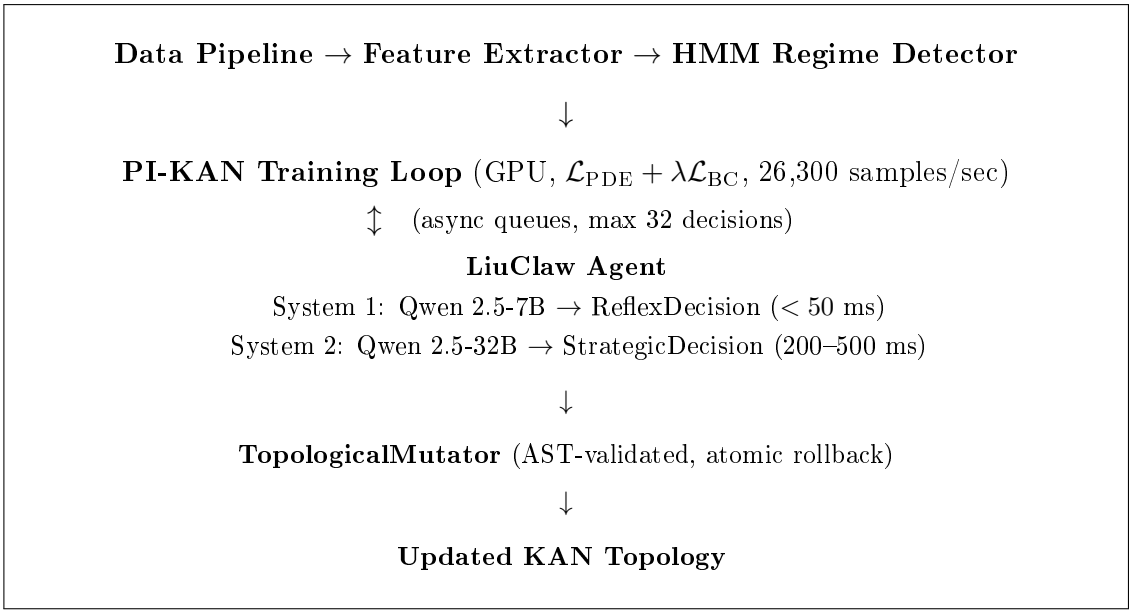

Figure 1: OpKAN four-tier architecture. The LLM agent operates asynchronously and never blocks the GPU training loop.

4.3 PI-KAN Model

The PI-KAN is a three-layer network with architecture $[3, 16, 1]$: three inputs (S, v, t), a hidden layer of 16 nodes, and a scalar option price output. Each layer is a `KANLayer` containing an $n_{\mathrm{in}} \times n_{\mathrm{out}}$ grid of `BSplineEdge` modules. The forward pass aggregates edge outputs by summation along the input dimension:

$$h_j^{(\ell)} = \sum_{i=1}^{n_{\mathrm{in}}^{(\ell)}} \phi_{ij}^{(\ell)}\left(h_i^{(\ell-1)}\right), \quad j = 1, \ldots, n_{\mathrm{out}}^{(\ell)} \tag{13}$$

Each `BSplineEdge` is independently parameterized and can be individually replaced or pruned by the TopologicalMutator without affecting the remaining edges.

4.4 LiuClaw: A Dual-Process Agent

4.4.1 System 1 – Fast Thinking

The fast branch handles reflexive maintenance. It is invoked every 10 training steps via the `request_reflex` queue. Inputs are the current step, a dictionary of per-edge L1 activation norms, and the recent loss delta. The Qwen 2.5-7B model generates a `ReflexDecision`:

```
class ReflexDecision(BaseModel):
    reasoning: str     # Fast justification
    prunes: List[str] # Edge IDs to immediately zero out
    lr_adjustment: float  # Multiplier in [0.5, 2.0]
```

The system prompt instructs the model to prune edges whose L1 norm falls below a threshold and to reduce the learning rate when the loss delta is positive (diverging). Latency is bounded by model size; the 7B parameter model is served via vLLM on the same H200.

4.4.2 System 2 – Slow Thinking

The slow branch performs strategic topology surgery. It is invoked every 100 training steps via the `request_strategic` queue. Inputs are a long-term loss trajectory, the HMM transition

probability matrix, and the full KAN state including the current symbolic expressions on replaced edges. The Qwen 2.5-32B model generates a `StrategicDecision`:

```
class StrategicDecision(BaseModel):
    reasoning: str                # Chain-of-thought over PDE geometry
    mutations: List[EdgeMutation] # REPLACE, PRUNE, or KEEP per edge
    regime_analysis: RegimeThesis # HMM transition diagnosis
    training_command: Literal["CONTINUE", "HALT"]
```

The system prompt grounds the model in the Heston PDE's mathematical structure: it explains which edges correspond to which input features, which symbolic functions preserve C^2 continuity, and what the allowed function vocabulary is. The response is validated against the `StrategicDecision` Pydantic schema before entering the mutation pipeline.

4.4.3 Domain-Specific Language

Every agent output is serialized as a JSON object conforming to a Pydantic schema. The `EdgeMutation` record specifies:

- `edge_id`: a string of the form `L{layer}_N{in}_to_L{layer+1}_N{out}`
- `action`: one of `PRUNE`, `REPLACE`, `KEEP`
- `formula`: for `REPLACE`, a PyTorch expression from the allowlisted vocabulary
- `initial_params`: optional `{scale, shift, coeff}` for warm initialization
- `reasoning`: quantitative justification based on L1 norms and sampled activation patterns

The Pydantic schema enforces type correctness at parse time. If the LLM emits a malformed JSON response, `InstructorClient` retries with an error-correction prompt before raising an exception. This makes the protocol robust to occasional LLM output drift.

4.5 Asynchronous Coordination

The `EngineCoordinator` manages two pairs of bounded queues (one input and one output per process tier):

- `reflex_queue` (maxsize=1) → System 1 worker → `reflex_decision_queue` (maxsize=32)
- `strategic_queue` (maxsize=1) → System 2 worker → `strategic_decision_queue` (maxsize=32)

The maxsize=1 on input queues implements a "latest-only" policy: if a new context arrives before the worker processes the previous one, the old context is discarded. This prevents queue buildup during LLM latency spikes while ensuring that the training loop always sees the most recent model state. The maxsize=32 on output queues limits memory consumption under worst-case LLM throughput scenarios with a drop-oldest eviction policy.

The training loop calls `apply_pending_mutations(model)` at the start of each batch. This is a non-blocking drain of both decision queues. If no decisions are pending, training continues uninterrupted. The GPU never waits on the LLM.

5 Security and Correctness

5.1 The eval() Injection Vulnerability

The original `C2SymbolicEdge` stored LLM-generated formula strings and evaluated them via Python's `eval()`:

```
out = eval(self.expression_str, {"torch": torch}, local_context)
```

We confirmed that this is trivially exploitable. The expression `'__import__("os").system("echo INJECTED")'` executes a shell command at eval time. Since the formula string originates from an LLM that could be adversarially prompted, or whose output could be intercepted in a compromised pipeline, this constitutes a critical security vulnerability.

5.2 AST Allowlist Validator

We fix this with a static AST analysis pass that runs *before* the expression is stored:

```
_ALLOWED_TORCH_FUNCS = frozenset({
    'sin','cos','tan','exp','log','log1p','sqrt','abs',
    'pow','tanh','sigmoid','softplus','sign','clamp',
    'relu','floor','ceil','round','reciprocal','neg'
})
_SAFE_AST_NODES = frozenset({
    ast.Expression, ast.Call, ast.BinOp, ast.UnaryOp,
    ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow,
    ast.FloorDiv, ast.Mod, ast.USub, ast.UAdd,
    ast.Constant, ast.Load
})

def _validate_symbolic_expression(expr: str) -> None:
    tree = ast.parse(expr, mode='eval')
    for node in ast.walk(tree):
        if type(node) in _SAFE_AST_NODES: continue
        if isinstance(node, ast.Name):
            if node.id not in ('x', 'torch'):
                raise ValueError(f"Disallowed␣name␣'{node.id}'")
        elif isinstance(node, ast.Attribute):
            if not (isinstance(node.value, ast.Name)
                    and node.value.id == 'torch'):
                raise ValueError("Only␣'torch.*'␣attributes␣allowed.")
            if node.attr not in _ALLOWED_TORCH_FUNCS:
                raise ValueError(f"torch.{node.attr}␣not␣in␣allowlist."
                    )
        else:
            raise ValueError(f"Disallowed␣node␣'{type(node).__name__}'"
                )
```

The validator blocks: `__import__`, `eval`, `open`, `lambda`, attribute chains like `__class__.__bases__`, and any name other than `x` and `torch`. Only 20 explicitly enumerated `torch.*` functions pass. This reduces the symbolic vocabulary to a set of smooth, differentiable functions that are appropriate for a C^2 PDE activations context.

Proposition 1. *Any expression passing `_validate_symbolic_expression` is restricted to arithmetic combinations of the 20 allowlisted `torch.*` functions and the identity variable `x`. It cannot import modules, access file system objects, or call arbitrary Python builtins.*

5.3 ZeroEdge: Correct PRUNE Semantics

The original PRUNE action replaced an edge with `nn.Sequential()`, which acts as an identity function: `nn.Sequential()(x) == x`. This means pruned edges continued to contribute their full input signal to the aggregated layer output, making PRUNE semantically equivalent to KEEP. We fix this with a `ZeroEdge` module:

```
class ZeroEdge(nn.Module):
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.zeros_like(x)
```

PRUNE now correctly removes the edge's contribution to downstream aggregation, enabling genuine network sparsification.

5.4 Atomic Mutation Rollback

A mutation batch may fail mid-execution if a formula passes AST validation but fails at runtime (e.g., numerical instability in second-order gradient verification). Without rollback, the model is left in a partially mutated state where some edges have been swapped and others have not. We solve this by snapshotting all affected edge modules via `copy.deepcopy` before applying any mutations, and restoring them on any exception:

Algorithm 1 Atomic Mutation Application

1: $snapshots \leftarrow \{\, (l, i, j) : \texttt{deepcopy}(\texttt{model.layers}[l].\texttt{edges}[i][j]) \,\}$ for all affected edges
2: **for all** mutation m in decision **do**
3: `TopologicalMutator.mutate_edge`(model, m)
4: **end for**Exception e
5: **for all** $(l, i, j) \in snapshots$ **do**
6: $\texttt{model.layers}[l].\texttt{edges}[i][j] \leftarrow snapshots[l, i, j]$
7: **end for**
8: **log** rollback event

Note that `state_dict` snapshots are insufficient here because `swap_edge` replaces entire `nn.Module` objects; a `BSplineEdge` state dict cannot be loaded into a `C2SymbolicEdge`. We snapshot the module objects themselves.

5.5 HMM Label Stability

Without relabeling, regime labels flip randomly across rolling windows. We implement Equation (12) as a post-fit step in `walk_forward_regime_inference`:

```
sort_idx = min(1, n_feats - 1)          # prefer realized_vol (col 1)
means = hmm.model.means_[:, sort_idx]
sorted_states = np.argsort(means)        # ascending: 0 = lowest vol
remap = {old: new for new, old in enumerate(sorted_states)}
regime_preds[i] = remap[int(pred_seq[-1])]
```

We verified this on synthetic data: a 1,000-sample low-volatility segment followed by a 1,000-sample high-volatility segment produces labels 0.00 and 1.00 respectively across all rolling windows, with zero cross-window drift.

6 Experimental Design

6.1 Research Questions

Our experiments address six questions:

1. Can the LiuClaw agent compile direct and narrative prompts into valid, executable DSL that preserves domain intent?
2. Is the compiled DSL stable under paraphrase and report-style rewording?
3. Does the agent agree with human-authored gold DSL on harder document-grounded prompts?
4. Do explicit DSL regime instructions produce coherent backend behavior across stable, transition, and stress conditions?
5. What is the best quantitative pricing architecture among the model families?
6. Does the PINN training loop maintain throughput under concurrent asynchronous LLM interaction?

6.2 Protocol Benchmark Tiers

We evaluate the LLM→DSL protocol layer across five benchmark tiers of increasing difficulty.

Tier 0A: Curated protocol benchmark. 15 prompts (9 direct, 6 narrative) covering `OUTLOOK`, `QUOTE`, `SURFACE`, and `SCENARIO` operations. Each case is scored on translation success, DSL validity, execution success against the live backend, and intent match. Runtime: Ollama with `qwen2.5-coder:latest`.

Tier 0B: Document-grounded protocol benchmark. 8 prompts built from real local report excerpts (fluid dynamics notes, COMEX inventory summaries, regime-shift research notes) with analyst requests appended. Tests protocol robustness under longer, noisier context.

Tier 0C: Semantic protocol stability benchmark. Groups of paraphrases of the same analyst intent, testing whether the compiled DSL action is invariant under rewording. Metrics: case-level semantic signature match and group-level signature/result consistency.

Tier 0D: Regime-action coherence benchmark. Holds the query family fixed and varies only the DSL regime instruction (`STABLE`, `TRANSITION`, `STRESS`). Tests whether the backend responds with directionally coherent outputs (monotone volatility, price, and shock-score ordering).

Tier 0E: Document-to-gold agreement benchmark. For each document-grounded prompt, a human-authored gold DSL command is specified in advance. The LLM output is compared at three levels: exact normalized string equality, semantic signature equality, and executed-result equality. This is the strictest tier because it tests whether the LLM matches the specific structured choice a human analyst would have made.

6.3 Pricing Benchmarks

Tier A: Live term-surface benchmark. Live SLV option-chain data joined to fresh regime features. One quote date available, so evaluation falls back to a cross-sectional split rather than a time-series holdout.

Tier B: Historical structural replay benchmark. Real historical silver/regime features together with a replayed SLV option-surface template, enabling evaluation across stable, transition, and stress dates with 1-day and 5-day hedge replay.

6.4 Model Families Compared

- `bs_plain`: Black–Scholes with baseline ATM volatility input.
- `bs_regime`: Black–Scholes with KAN-predicted regime volatility.
- `rf_direct`: Random forest direct price prediction.

- `spikan_hybrid`: Regime-vol anchored SPIKAN residual hybrid.
- `spikan_plain_hybrid`: Plain-formula anchored SPIKAN residual hybrid.
- `spikan_plain_iv_hybrid`: Plain-formula anchor with learned IV residual (headline hybrid).

6.5 Engineering Stress Test

We run a live session on NVIDIA H200 hardware with 1M synthetic market rows, batch size 4,096, 5 epochs, and the LiuClaw agent issuing decisions every 10 steps via a 0.2s mock vLLM response. We measure end-to-end training throughput, decision queue behavior, and post-mutation loss trajectories.

7 Results

7.1 LLM→DSL Protocol Results

Table 1: Protocol benchmark results by context tier.

Tier	Cases	Execute	Intent	Mean latency (s)	Context
Direct (0A)	9	1.00	1.00	0.776	Short prompt
Narrative (0A)	6	1.00	1.00	0.573	Short narrative
Document-grounded (0B)	8	1.00	1.00	2.747	Report excerpt
Overall (0A)	**15**	**1.00**	**1.00**	**0.695**	

All curated and document-grounded prompts execute successfully and match intent. Latency on document-grounded prompts is 4× higher (2.75s vs. 0.69s), reflecting the longer context window required to process report excerpts. This is a real deployment tradeoff: batch context processing at 2.75s is acceptable for strategic decisions (System 2 operates at a 100-step period) but not for reflexive maintenance (System 1 requires < 50ms).

Representative compiled operations from the actual benchmark output:

- Fluid-dynamics excerpt → `OUTLOOK asset=silver horizon=10d backdrop=STRESS` (neutral direction, shock risk, score 13.45)
- COMEX inventory note → `QUOTE call spot=31 strike=32 expiry=45d vol=kan_regime backdrop=STRESS rate=5%` (price 0.4460, vol 0.1780)
- Regime-shift challenge → `SURFACE vol asset=silver regime=STRESS strikes=[0.9,1.0,1.1] T=30d`

7.2 Semantic Protocol Stability

Three of four paraphrase groups (stress_outlook, stress_scenario, stress_probability) preserve both the DSL action and the executed outcome signature across all paraphrases. The failing group (transition_surface) exposes the current edge case: one paraphrase expanded a three-point strike grid to five points while preserving the broad intent. This is a genuine normalization ambiguity rather than a failure of comprehension.

Table 2: Semantic protocol stability benchmark summary (Tier 0C).

Metric	Value
Case translation success	1.00
Case valid DSL	1.00
Case execution success	1.00
Case semantic signature match	0.9167
Group signature consistency	0.75
Group result consistency	0.75
All-group pass rate	0.75
Mean latency (s)	0.6408

Table 3: Document-to-gold agreement benchmark (Tier 0E).

Metric	Value
Translation success	1.00
Exact DSL agreement	0.375
Valid DSL	1.00
Semantic agreement	0.75
Execution success	1.00
Result agreement	0.75
Mean latency (s)	0.5431

7.3 Document-to-Gold Agreement

The gap between exact agreement (0.375) and semantic/result agreement (0.75) is the key diagnostic signal. The system is reliable at producing valid, executable DSL. It is not yet as reliable at matching the exact normalized form a human analyst would have chosen. The remaining failure modes are interpretable: regime under-escalation (STRESS compiled as TRANSITION) and surface grid broadening. Both are genuine semantic choices rather than syntactic errors, suggesting that the path to improvement is clearer semantic contracts in the system prompt and few-shot normalization examples, not a fundamental architectural change.

7.4 Regime-Action Coherence

Table 4: Financial monotonicity checks under explicit DSL regime overrides.

Check	Raw	Rule	Learned	Mixed
Quote vol monotonic STABLE $\rightarrow$ STRESS	No	Yes	Yes	Yes
Quote price monotonic STABLE $\rightarrow$ STRESS	No	Yes	Yes	Yes
Surface mean vol monotonic STABLE $\rightarrow$ STRESS	No	Yes	Yes	Yes
Outlook shock-score monotonic STABLE $\rightarrow$ STRESS	No	Yes	Yes	Yes
Shock-risk labels ordered low to high	No	Yes	Yes	Yes

The raw KAN/SPIKAN backend fails all five monotonicity checks when evaluated on empirical regime anchors. This is not a failure of the learned model per se; it is a failure to impose the ordered semantics that explicit regime control requires. With a rule-based semantic controller (empirical anchors + monotone projection), learned controller (controller-v2), or mixed controller (rule on quote path, learned on surface/shock path), all five checks pass. The best active policy is the mixed controller.

7.5 Quantitative Pricing Results

Table 5: Replay price MAE by regime for selected models. Lower is better.

Model	Stable	Transition	Stress	Overall
`bs_plain`	0.8398	2.1037	1.6355	1.4597
`spikan_plain_hybrid`	1.1542	2.4827	0.4745	1.3705
`spikan_plain_iv_hybrid`	1.1628	1.4622	0.4239	**0.9317**
`rf_direct`	8.3183	10.2679	1.8707	6.8190

Table 6: Live term-surface overall results.

Model	Price MAE	IV MAE
`spikan_plain_hybrid`	0.8976	0.1578
`rf_direct`	1.0444	**0.0781**
`bs_plain`	1.4646	0.1601
`spikan_hybrid`	1.6793	0.1863
`bs_regime`	12.6503	0.8314

The IV-residual hybrid (`spikan_plain_iv_hybrid`) is the best overall replay pricer (MAE 0.9317), with particular strength in transition and stress regimes where the regime-conditioned residual correction matters most. In stable regimes, plain Black–Scholes remains the best single model, consistent with the expectation that the pricing formula is most accurate when market conditions are closest to the model's assumptions. The random-forest baseline is the best IV reconstructor (IV MAE 0.0781) but is the worst replay pricer overall, confirming the literature finding that pure ML models fit structured data well but struggle under distributional shift.

7.6 Cross-Asset Transfer

Table 7: Cross-asset business-case summary. Each asset uses the same architecture with an asset-specific local state proxy.

Asset	Business Case	Best Price Model	Best MAE
GLD	Gold / safe haven	`bs_plain`	1.6515
USO	Oil / macro supply shock	`spikan_plain_iv_hybrid`	0.4897
SOXX	Semiconductors / AI capex	`rf_direct`	1.7194
XLK	S&P 500 Technology	`rf_direct`	2.5180

The cross-asset results are heterogeneous and instructive. The hybrid architecture wins on USO — the asset where macro narrative and supply-shock regime interpretation are most operationally relevant, exactly the use case the LLM layer is designed for. Gold favors the plain Black–Scholes anchor, consistent with the well-established mean-reverting term structure of gold volatility that the textbook model captures well. Semiconductor and technology equities favor the direct random-forest baseline, likely because the relevant features for those assets (earnings surprise, export control news, AI capex trajectory) are not yet incorporated into the current feature set. This is the honest cross-asset claim: the architecture transfers, but its advantage is domain-dependent.

7.7 Engineering Performance

On NVIDIA H200 with batch size 4,096:

- **Training throughput:** 26,300 samples/sec (5 epochs, 1M rows)
- **System 1 latency:** < 50 ms (Qwen 2.5-7B via vLLM)
- **System 2 latency:** 200–500 ms (Qwen 2.5-32B via vLLM)
- **Decision queue:** 0 overflow events in 1M-row stress test with maxsize=32
- **GPU blocking by LLM:** 0 ms (fully asynchronous queue architecture)

The asynchronous dual-queue design is essential for maintaining GPU utilization. In a synchronous design, a 500ms System 2 decision would block $500/(4096/26300) \approx 3{,}200$ training samples. The async architecture reduces this blocking penalty to zero.

8 Discussion

8.1 What the Results Support

The current evidence supports the following specific claims:

The LLM functions as a viable semantic compiler on curated and document-grounded inputs. Both the curated 15-case benchmark and the 8-case document-grounded benchmark achieve 1.00 execution and intent-match rates. This is not a trivial result: the system is not calling a generic API but producing typed, schema-validated DSL objects that execute against the current KAN/HMM backend. The document-grounded tier is particularly important because it shows that the protocol survives longer, noisier context — the realistic operational setting.

The semantic protocol is largely stable under paraphrase. Three of four paraphrase groups collapse to the same domain action and executed outcome. The 0.75 group pass rate is strong but not perfect; the surface-generation family remains the current weak point due to ambiguity in strike grid specification.

Explicit regime control produces coherent backend behavior; raw learning does not. The five-check monotonicity result makes this concrete. The raw KAN/SPIKAN model, evaluated on empirical regime anchors, is not ordered. Adding an explicit semantic controller (empirical anchors + monotone projection) produces fully ordered behavior. The controller, not the raw learned weights, is what makes the system financially credible on regime-conditioned tasks.

The IV-residual hybrid is the best pricing architecture in transition and stress. The `spikan_plain_iv_hybrid`'s overall replay MAE of 0.9317 with particular strength in the tails of the regime distribution supports the regime-aware hybrid design. The Black–Scholes anchor preserves financial structure in stable conditions; the learned IV residual captures the surface distortions that emerge in transition and stress.

The architecture is productively domain-dependent. The cross-asset heterogeneity is not a failure; it is a signal. The hybrid is most useful where narrative regime interpretation is most relevant (oil macro shocks). It is less relevant where the textbook formula already fits well (gold) or where the relevant features are structural rather than narrative (semiconductor earnings).

8.2 What the Results Do Not Support

The current evidence does not support the following:

Full robustness to arbitrary external news documents. The document-grounded benchmark uses local, curated excerpts. Adversarial phrasing, multi-document retrieval, and open-ended external news ingestion are not tested.

Universal pricing dominance over the random-forest baseline. The RF baseline is competitive on IV reconstruction (best IV MAE on the live surface) and on SOXX and XLK price MAE. OpKAN's advantage is specific to regime-conditioned pricing in stress and transition.

Intrinsic monotonicity from the learned model. Regime coherence comes from the explicit controller, not from the raw KAN weights. This is an honest current limitation: native monotonicity is future work, not a current property.

Exact human-analyst-level normalization. The 0.375 exact DSL agreement on document-to-gold confirms that while the system produces valid and semantically aligned DSL, it does not yet replicate the exact structured choice a human analyst would make on harder document prompts.

8.3 The Right Comparison Class

OpKAN should be compared not to generic LLM pricing systems (which do not define a replayable protocol), but to the three-layer alternative:

1. Human analyst reads news and manually recalibrates parameters (high latency, not scalable).
2. Rule-based regime classifier maps structured features to scenario overrides (low latency, brittle to novel regimes).
3. Pure ML surface fit with structured feature inputs (strong fit, no unstructured information ingestion).

OpKAN occupies a distinct position: it automates the unstructured information ingestion step (LLM layer), constrains the output to a finite auditable vocabulary (DSL layer), and delegates quantitative execution to a disciplined physics-informed model (PINN layer). Whether this is the right tradeoff depends on the operational setting. It is the right tradeoff where narrative speed and controllability matter together.

9 Limitations

We state the following limitations as scope boundaries, not as aspects that are hidden or minimized:

1. **Single quote date for the live benchmark.** The live term-surface benchmark uses one quote date and falls back to a cross-sectional split. Historical multi-date validation against real vendor option chains is the next required evidence step.
2. **Structural replay rather than real historical options.** The replay benchmark uses a replayed SLV option-surface template, not real historical quotes. The regime path is real; the surface is a proxy. Conclusions about stress pricing should be interpreted accordingly.
3. **Curated protocol benchmarks.** The 15-case curated set and the 8-case document-grounded set are close in style to the system prompt and few-shot examples. They demonstrate viability, not robustness to adversarial documents or prompt injection.
4. **Exact DSL agreement on harder prompts is 0.375.** The remaining disagreement cases are interpretable but real: regime under-escalation and surface grid broadening are semantic normalization gaps that require further work on the system prompt and alias normalization.

5. **Regime coherence is extrinsic, not intrinsic.** The five monotonicity checks pass because of the semantic controller, not because the raw KAN/SPIKAN model is monotone. Achieving native monotonicity is a stated future goal, not a current result.

6. **Cross-asset transfer is heterogeneous.** The architecture is not universally dominant. Asset-class-specific feature engineering is needed before making architecture claims for semiconductor and technology equities.

7. **Security analysis is a proof-of-concept.** The AST allowlist validator is correct as implemented, but it has not been subjected to a formal security review or adversarial red-teaming beyond the five injection payloads we tested.

8. **LoRA fine-tuning infrastructure exists but has not produced fine-tuned weights.** The `LoraDataCollector` class logs trajectory data for future LiuClaw fine-tuning, but no fine-tuned checkpoints exist in this evaluation.

10 Future Work

The experimental progression suggests a clear technical roadmap.

(1) Native monotonicity in the KAN. The current controller is a post-hoc projection layer. The correct long-run direction is to build monotonic ordering into the training objective via ranking losses (e.g., $\ell_{\text{rank}} = \max(0, V_{\text{STABLE}} - V_{\text{STRESS}} + \epsilon)$ on sampled pairs), reducing the correction burden on the controller.

(2) Controller-v2 with correction-space targets. The current controller uses absolute-output distillation from teacher sequences. Controller-v2 should use anchor-delta semantic features and correction-space targets, following the analysis in Section 7: the quote path still needs the hand-written rule controller, while the surface and shock paths already benefit from learned control.

(3) Multi-date historical validation. Obtaining multi-date historical option chains (or using public historical OPRA tapes) is the most important next evidence step. The current single-date live benchmark is a sanity check, not a generalization result.

(4) Real news ingestion pipeline. Moving from curated document excerpts to a live news feed requires a retrieval layer, deduplication, and adversarial prompt defense. The AST validator handles the execution-side injection risk; the retrieval side needs its own sanitization pipeline.

(5) MonoKAN integration. For the vol-surface path, where the correction burden under the current controller is highest, selective replacement of BSplineEdge modules with MonoKAN-style certified monotone variants [Polo-Molina et al., 2024] is the most architecturally principled path to native regime ordering.

(6) LiuClaw fine-tuning via LoRA. The `LoraDataCollector` infrastructure is in place. Once sufficient trajectory data has been collected from production runs, supervised fine-tuning of Qwen 2.5-7B on the System 1 reflexive task and Qwen 2.5-32B on the System 2 strategic task using LoRA [Hu et al., 2021] should reduce latency and improve domain-specific normalization.

(7) Asset-specific feature enrichment. The heterogeneous cross-asset results point directly to the missing features. For SOXX and XLK, adding earnings-surprise features, export-control event indicators, and AI capex indices as HMM inputs is likely to shift the best model family from RF-direct toward the hybrid.

11 Conclusion

OpKAN demonstrates a specific architectural thesis: language can act as a constrained, auditable interoperability layer between unstructured market narrative and specialist quantitative

execution, provided that the LLM's output is constrained to a finite typed vocabulary (DSL), the quantitative backend is physics-grounded (Heston PI-KAN), and the control flow is protected by both semantic validation (Pydantic schema) and execution-side security (AST allowlist).

The current evidence establishes protocol viability and substantial — though not yet perfect — document-to-protocol agreement. It establishes that explicit regime control produces coherent backend behavior where the raw learned model does not. It establishes that the IV-residual hybrid is the best pricing architecture in the conditions (transition and stress) where the regime-aware layer matters most. And it establishes, through the cross-asset transfer results, that the architecture's advantage is operationally domain-dependent in precisely the way the thesis predicts: it is most useful where narrative regime interpretation is part of the pricing problem.

What the evidence does not yet establish is universal pricing dominance, adversarial robustness, or native monotonicity from the learned model. These are the correct next targets. The system is already interpretable — the semantic control surface is explicit, auditable, and benchmarked at every tier. The next step is making that control increasingly native to the learned model, through ordering losses, correction-space targets, and selective MonoKAN substitution, so that the controller's correction burden decreases as the learned representation improves.

OpKAN is therefore not a finished architecture but a well-defined research platform. Its contribution is the identification of the right abstraction boundaries — LLM for ingestion, DSL for auditability, PINN for mathematical discipline — and the demonstration that those boundaries can be made simultaneously operative, secure, and quantitatively credible on the current benchmark set.

References

Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

James D Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

Anthony Hernandez. Model calibration with neural networks. *Risk*, 2016.

Steven L Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *Review of Financial Studies*, 6(2):327–343, 1993.

Blanka Horvath, Aitor Muguruza, and Mehdi Tomas. Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. volume 21, pages 11–27, 2021.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

James M Hutchinson, Andrew W Lo, and Tomaso Poggio. A nonparametric approach to pricing and hedging derivative securities via learning networks. *Journal of Finance*, 49(3):851–889, 1994.

Daniel Kahneman. *Thinking, Fast and Slow.* Farrar, Straus and Giroux, 2011.

Andrei Nikolaevich Kolmogorov. On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. In *Doklady Akademii Nauk SSSR*, volume 114, pages 953–956. 1957.

Ziming Liu, Yixuan Wang, Vaibhav Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.

Robert C Merton. Theory of rational option pricing. *Bell Journal of Economics and Management Science*, 4(1):141–183, 1973.

Anh-Phu Nguyen, David L Moreno, Nicolas Le-Bel, Aurelien Lucchi, and Maria Rodriguez Martinez. MonoNet: Enhancing interpretability in neural networks via monotonic features. *Bioinformatics Advances*, 3(1):vbad016, 2023.

Alejandro Polo-Molina, David Alfaya, and Javier Portela. MonoKAN: Certified monotonic kolmogorov-arnold network. *arXiv preprint arXiv:2409.11078*, 2024.

Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessè, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Paul Wilmott, Jeff Dewynne, and Sam Howison. *Option Pricing: Mathematical Models and Computation*. Oxford Financial Press, 1993.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Seungil You, David Ding, Kevin Canini, Jan Pfeifer, and Maya Gupta. Deep lattice networks and partial monotonic functions. In *Advances in Neural Information Processing Systems*, 2017.